

1. Título del laboratorio

Laboratorio 02: Integración de Signaling, Saltos No Locales y Message Queues de IPC

2. Objetivo general

Implementar un mecanismo de comunicación entre procesos en Linux que integre señalizaciones (signals), saltos no locales mediante `sigsetjmp()`/`siglongjmp()` y colas de mensajes IPC (Message Queues).

El laboratorio consiste en desarrollar dos procesos independientes:

- Un proceso emisor/receptor de señales, que recibe señales desde la línea de comandos, identifica el tipo de señal recibida, realiza un salto no local y posteriormente envía un mensaje a una cola de mensajes IPC.
- Un proceso consumidor, que permanece ejecutándose en un ciclo y recibe los mensajes enviados mediante la cola IPC, procesándolos según su tipo.

Se utilizarán tres tipos de señalización, asociados a tres tipos de mensajes:

Señalización	Tipo de mensaje
Tipo 1	1
Tipo 2	2
Tipo 3	3

El proceso receptor no deberá finalizar después de procesar una señal y deberá permanecer disponible para recibir nuevas señalizaciones.

3. Objetivos específicos

Al finalizar el laboratorio, el estudiante será capaz de:

1. Enviar una señal a un proceso utilizando la línea de comandos.
2. Identificar diferentes tipos de señales recibidas por un proceso.
3. Implementar manejadores para tres tipos de señalización.
4. Utilizar `sigsetjmp()` y `siglongjmp()` para realizar saltos no locales.
5. Crear y utilizar una Message Queue de IPC para la comunicación entre procesos.
6. Asociar cada señal recibida con un tipo de mensaje.
7. Enviar mensajes mediante una cola IPC desde un proceso a otro.
8. Recibir y procesar mensajes desde una cola IPC.
9. Implementar procesos que permanezcan activos mediante ciclos de ejecución.

10. Comprender la interacción entre mecanismos de señalización y comunicación entre procesos.

4. Recursos y herramientas necesarias

Sistema operativo

- GNU/Linux.
- WSL2, máquina virtual Linux o instalación nativa compatible con POSIX.

Herramientas

- GCC.
- Terminal de comandos.
- Editor de texto o IDE.

Bibliotecas y mecanismos

El código proporcionado por el profesor utilizará los mecanismos necesarios para:

- Signals.
- `sigsetjmp()` / `siglongjmp()`.
- IPC Message Queues.
- Control de procesos.
- Suspensión temporal mediante `sleep()`.

Requisito previo

El estudiante debe tener conocimientos básicos de:

- Procesos en Linux.
- PID.
- Señales.
- Manejo básico de archivos fuente en C.
- Compilación de programas mediante GCC.
- Concepto básico de comunicación entre procesos.

5. Introducción breve

Los sistemas operativos proporcionan diferentes mecanismos para permitir la comunicación y coordinación entre procesos.

Las señales (signals) permiten notificar a un proceso que ha ocurrido un determinado evento. Una señal puede ser enviada desde otro proceso utilizando herramientas de línea de comandos.

En este laboratorio se utilizarán tres tipos de señalización. Cada señal recibida deberá provocar una acción específica dentro del proceso receptor.

Después de identificar la señal, el proceso realizará un salto no local utilizando `siglongjmp()`, regresando a un punto de recuperación establecido previamente mediante `sigsetjmp()`.

Una vez recuperado el flujo de ejecución, el proceso deberá generar y enviar un mensaje mediante una Message Queue de IPC.

La información enviada deberá identificar el tipo de señal recibida:

Señal 1 → Mensaje tipo 1

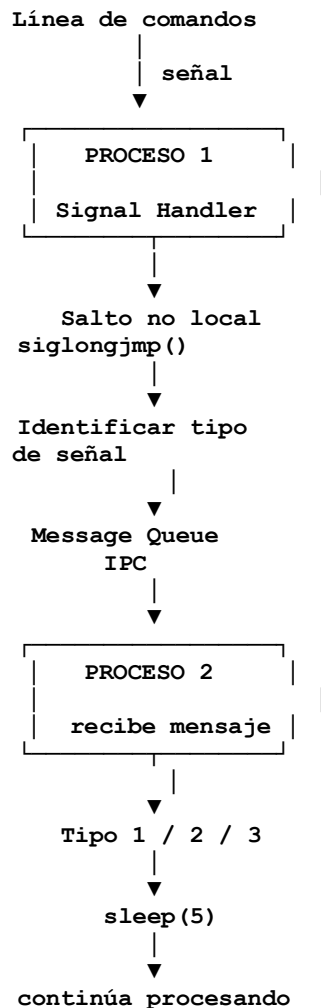
Señal 2 → Mensaje tipo 2

Señal 3 → Mensaje tipo 3

Un segundo proceso tendrá la responsabilidad de recibir los mensajes desde la cola IPC.

Este segundo proceso deberá permanecer ejecutándose en un ciclo y procesar los mensajes de acuerdo con su tipo.

El flujo general será:



6. Instrucciones del laboratorio

Paso 1 — Preparar el entorno

Crear un directorio de trabajo para el laboratorio y copiar en él los dos archivos fuente proporcionados por el profesor.

Los archivos serán:

`emisor.c`
`receptor.c`

No se debe modificar la estructura general proporcionada por el profesor salvo en los puntos indicados durante el laboratorio.

Paso 2 — Identificar la arquitectura del laboratorio

Determinar cuál de los dos programas corresponde a:

- Proceso 1: recepción de señales y envío de mensajes.
- Proceso 2: recepción y procesamiento de mensajes.

Identificar también:

- PID del proceso que recibirá las señales.
- Identificador de la Message Queue.
- Manejadores de señales disponibles.
- Puntos de recuperación establecidos mediante `sigsetjmp()`.

Paso 3 — Compilar los programas

Compilar ambos programas utilizando GCC y las opciones de advertencia apropiadas.

Verificar que ambos programas compilen correctamente antes de iniciar las pruebas.

No deberán existir errores de compilación.

Paso 4 — Ejecutar el proceso consumidor

Ejecutar primero el programa encargado de recibir los mensajes.

Este proceso deberá permanecer en ejecución en un loop infinito, esperando mensajes provenientes de la Message Queue.

El proceso deberá estar preparado para recibir mensajes de tipo:

1
2
3

Paso 5 — Ejecutar el proceso receptor de señales

Ejecutar el segundo programa.

Este proceso deberá mostrar su PID, de manera que pueda ser utilizado desde otra terminal para enviarle señales.

El proceso deberá permanecer ejecutándose y esperando nuevas señalizaciones.

Paso 6 — Enviar la primera señal

Desde otra terminal, enviar al proceso la primera señal definida por el profesor.

El proceso deberá:

1. Recibir la señal.
2. Ejecutar el `signal handler` correspondiente.
3. Realizar un salto no local mediante `siglongjmp()`.

- 4.**Recuperar el flujo de ejecución mediante el punto establecido con `sigsetjmp()`.
- 5.**Determinar que corresponde al tipo de mensaje 1.
- 6.**Enviar un mensaje de tipo 1 mediante la Message Queue.
- 7.**Continuar ejecutándose.

El proceso no debe finalizar después de realizar estas operaciones.

Paso 7 — Enviar la segunda señal

Repetir el procedimiento utilizando la segunda señal.

El proceso deberá:

- 1.**Recibir la señal.
- 2.**Ejecutar su handler correspondiente.
- 3.**Realizar el segundo salto no local.
- 4.**Recuperar el flujo de ejecución.
- 5.**Generar un mensaje de tipo 2.
- 6.**Enviar el mensaje mediante la Message Queue.
- 7.**Continuar ejecutándose.

Paso 8 — Enviar la tercera señal

Enviar la tercera señal al proceso.

El proceso deberá:

- 1.**Recibir la señal.
- 2.**Ejecutar el tercer handler.
- 3.**Realizar el tercer salto no local.
- 4.**Recuperar el flujo de ejecución.
- 5.**Generar un mensaje de tipo 3.
- 6.**Enviar el mensaje mediante la Message Queue.
- 7.**Continuar ejecutándose.

Paso 9 — Verificar el procesamiento de mensajes

Observar el segundo proceso.

Cada vez que reciba un mensaje deberá:

- 1.**Extraer un mensaje de la Message Queue.
- 2.**Identificar su tipo.

3.Mostrar el tipo recibido.

4.Procesar el mensaje correspondiente.

5.Realizar un `sleep` de 5 segundos.

6.Continuar con el siguiente mensaje.

El proceso deberá permanecer ejecutándose indefinidamente.

Por ejemplo, si los mensajes llegan en el siguiente orden:

1 → 2 → 3 → 1 → 3 → 2

el proceso consumidor deberá procesarlos respetando el orden en que sean extraídos de la cola y realizar la espera de 5 segundos después de cada procesamiento.

Paso 10 — Realizar una prueba completa

Realizar una secuencia de pruebas utilizando las tres señalizaciones varias veces.

Verificar que:

- Los tres handlers funcionen correctamente.
- Se produzca un salto no local para cada señal.
- Cada señal genere el tipo de mensaje correspondiente.
- Los mensajes lleguen correctamente a la Message Queue.
- El segundo proceso pueda extraer los mensajes.
- Los mensajes de tipo 1, 2 y 3 sean identificados correctamente.
- El proceso que recibe las señales no termine después de procesarlas.
- El proceso que recibe los mensajes permanezca en ejecución.
- Ambos procesos puedan continuar trabajando durante múltiples ciclos.

7. Ejercicios prácticos

Ejercicio 1 — Señalización y salto no local

Completar el programa proporcionado por el profesor para implementar tres handlers, uno para cada tipo de señal.

Cada handler deberá estar asociado con su correspondiente punto de recuperación.

El estudiante deberá demostrar que:

Señal
↓
Handler
↓
`siglongjmp()`
↓
`sigsetjmp()`
↓
Continuación del proceso

Ejercicio 2 — Asociación señal-mensaje

Implementar la correspondencia entre señalización y Message Queue:

Señal	Tipo de mensaje
Señal 1	1
Señal 2	2
Señal 3	3

Después de recuperar el flujo mediante el salto no local, el proceso deberá enviar el mensaje correspondiente.

Ejercicio 3 — Recepción de mensajes

Completar el segundo programa para que permanezca en un ciclo infinito y extraiga mensajes de la Message Queue.

Para cada mensaje recibido deberá:

- identificar su tipo;
- mostrar información sobre el mensaje;
- procesarlo;
- esperar 5 segundos;
- continuar con el siguiente mensaje.

Ejercicio 4 — Prueba de integración

Realizar una secuencia de al menos seis señalizaciones, utilizando los tres tipos de señal.

Por ejemplo:

Señal 1
Señal 2
Señal 3
Señal 1
Señal 3
Señal 2

Registrar la salida obtenida por ambos procesos y verificar que exista correspondencia entre:

señal recibida → salto no local → mensaje enviado → mensaje recibido.

8. Criterios de evaluación

Criterio	Ponderación	Descripción
Manejo de señales	20%	Implementación correcta de los tres handlers y recepción de las señalizaciones.
Salto no locales	20%	Uso correcto de <code>sigsetjmp()</code> y <code>siglongjmp()</code> para recuperar el flujo de ejecución.
Message Queue IPC	25%	Creación/uso correcto de la cola, envío y recepción de mensajes.
Correspondencia señal-mensaje	15%	Asociación correcta entre los tres tipos de señal y los mensajes 1, 2 y 3.
Procesamiento de mensajes	10%	Lectura del tipo de mensaje, espera de 5 segundos y continuación del procesamiento.

Continuidad de los procesos	10%	Los procesos permanecen activos y pueden procesar múltiples señalizaciones y mensajes.
Total	100%	

9. Entregables

El estudiante deberá entregar dos archivos fuente en C:

`emisor.c`
`receptor.c`

Los archivos deberán:

- Compilar correctamente.
- Implementar los tres tipos de señalización.
- Implementar los tres saltos no locales.
- Utilizar la Message Queue de IPC.
- Asociar los mensajes 1, 2 y 3 con las respectivas señalizaciones.
- Mantener ambos procesos en ejecución.
- Permitir procesar múltiples señales y mensajes.

Importante: la solución deberá funcionar de manera continua; la recepción de una señal o el procesamiento de un mensaje no debe provocar la terminación de ninguno de los dos procesos.